TeamBridge - корпоративный сервис для обмена мгновенными сообщениями

Курс «System Design»

2. О себе

Мордвинов Дмитрий
Архитектор

3. Цель и задачи проекта

Цель — спроектировать архитектуру корпоративного сервиса для обмена мгновенными сообщениями в режиме реального времени внутри компании.

Задачи

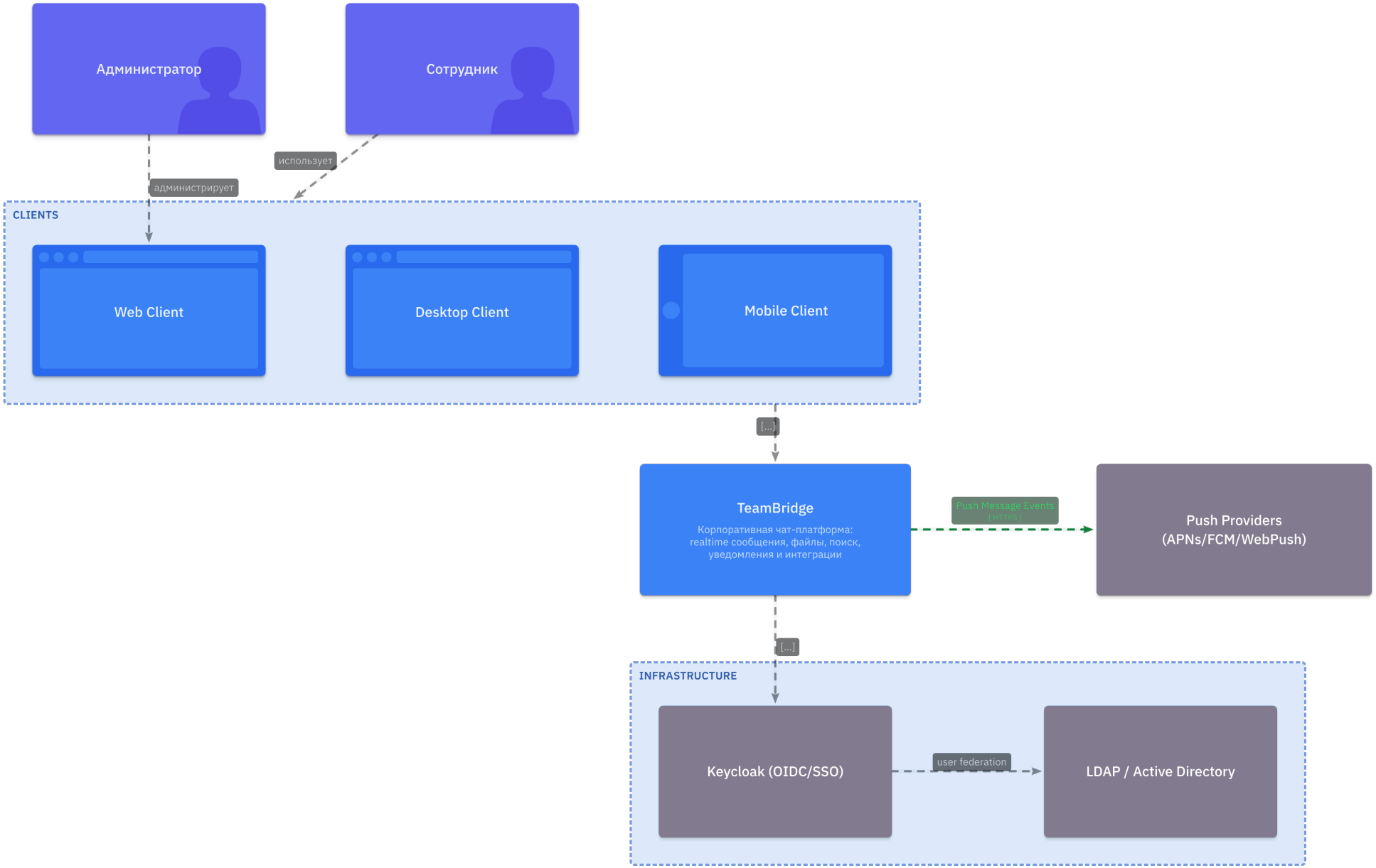
1. Зафиксировать функциональные и нефункциональные требования, риски.
2. Описать контекст, контейнеры, потоки данных, модель данных, API (OpenAPI), хранение (PostgreSQL, keyDB).
3. Спроектировать ИБ системы (OAuth 2.0 / OIDC / JWT, RBAC, Vault).
4. Спроектировать мониторинг (метрики, логи, трассировка).
5. Задать развёртывание IaC (Terraform, Ansible), контейнеры и Kubernetes, CI/CD.

4. Требования

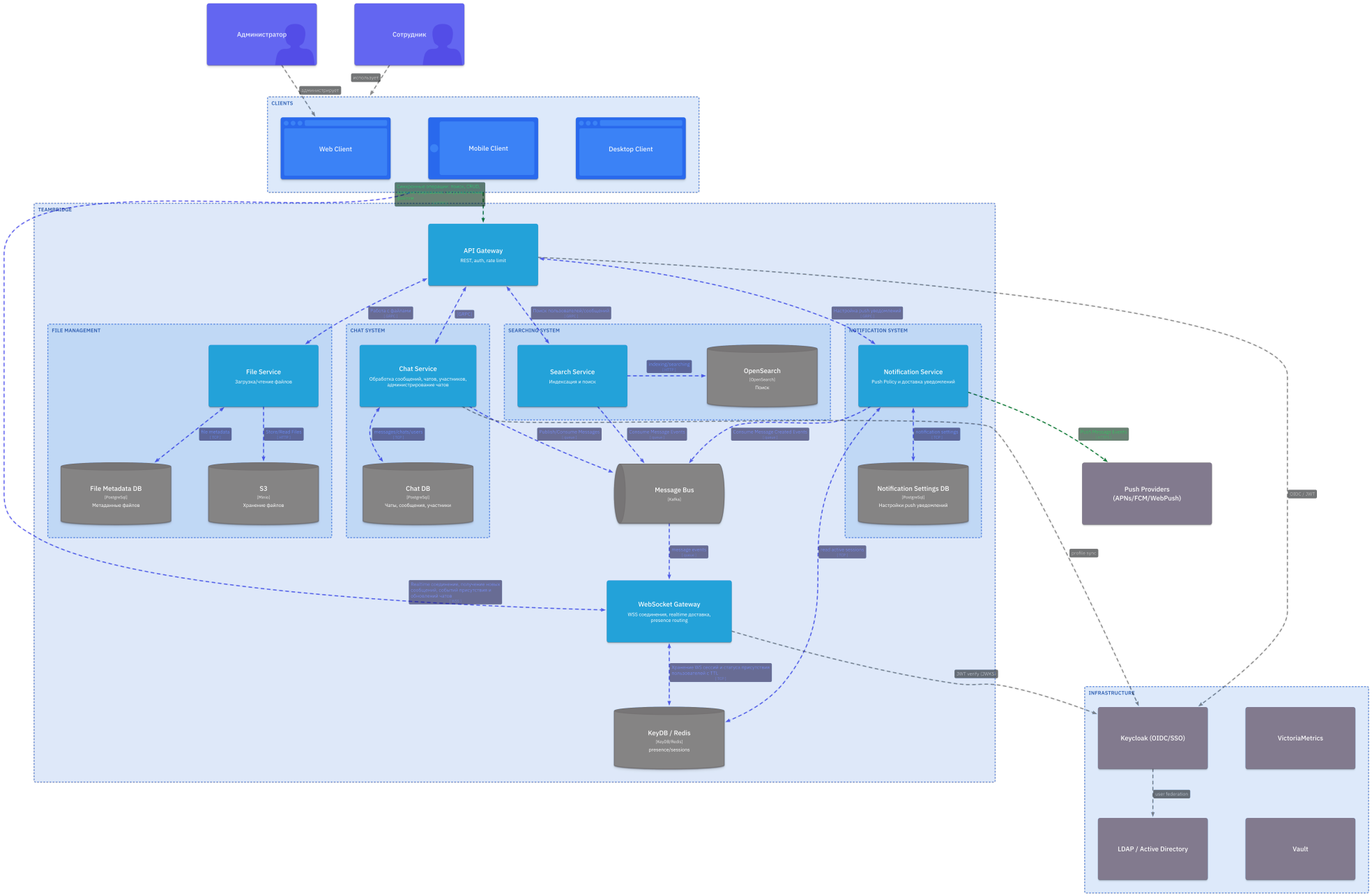
- Realtime-обмен сообщениями (личные и групповые чаты, включая режим каналов)
- Поиск по сообщениям и пользователям
- Файлообмен (до 100 МБ) и push-уведомления
- SSO + LDAP/AD, роли, аудит
- Отказоустойчивость и масштабирование (SLA 99.9%)
  - **Целевые метрики** - latency доставки до 300 мс, RPO до 5 мин, RTO до 30 мин
  - Защита трафика - TLS 1.3

5. Контекст

Система взаимодействует с пользователями (веб, мобильное приложение, desktop), провайдерами уведомлений, keycloak; внутренние сервисы изолированы за API Gateway.


Администратор
Сотрудник
использует
администрирует
CLIENTS
Web Client
Desktop Client
Mobile Client
[...]
TeamBridge
Корпоративная чат-платформа:
realtime сообщения, файлы, поиск,
уведомления и интеграции
Push Message Events
[gRPC]
Push Providers
(APNs/FCM/WebPush)
[...]
INFRASTRUCTURE
Keycloak (OIDC/SSO)
user federation
LDAP / Active Directory

6. Контейнерная диаграмма


Администратор
Сотрудник
используют
администрируют
CLIENTS
Web Client
Mobile Client
Desktop Client
TEAMBRIDGE
API Gateway
REST, auth, rate limit
Работа с файлами
gRPC
Поиск пользователей/сообщений
Настройка push уведомлений
FILE MANAGEMENT
File Service
Загрузка/чтение файлов
file metadata
store/Read Files
File Metadata DB
[PostgreSql]
Метаданные файлов
S3
[Minio]
Хранение файлов
CHAT SYSTEM
Chat Service
Обработка сообщений, чатов, участников, администрирование чатов
messages/chats/users
Chat DB
[PostgreSql]
Чаты, сообщения, участники
SEARCHING SYSTEM
Search Service
Индексация и поиск
indexing/searching
OpenSearch
[OpenSearch]
Поиск
NOTIFICATION SYSTEM
Notification Service
Push Policy и доставка уведомлений
notification settings
Notification Settings DB
[PostgreSql]
Настройки push уведомлений
Publish/Consume Messages
Consume Message Events
Consume Message Created Events
Message Bus
[Kafka]
message events
read active sessions
Push Providers
(APNs/FCM/WebPush)
profile sync
OIDC / JWT
Реальное соединение, получение новых сообщений, событий присутствия и обновлений чатов
WebSocket Gateway
WSS соединения, realtime доставка, presence routing
Хранение WS сессий и статуса присутствия пользователей с TTL
KeyDB / Redis
[KeyDB/Redis]
presence/sessions
JWT verify (JWKS)
INFRASTRUCTURE
Keycloak (OIDC/SSO)
VictoriaMetrics
user federation
LDAP / Active Directory
Vault

7. Архитектурные подходы

- Микросервисная архитектура - упрощает независимое масштабирование и развитие отдельных доменов
- Event Sourcing + событийное взаимодействие через Kafka - надежная асинхронная обработка, replay, слабая связанность
- Разделение API Gateway и WebSocket Gateway - оптимизация под sync API и long-lived realtime-соединения

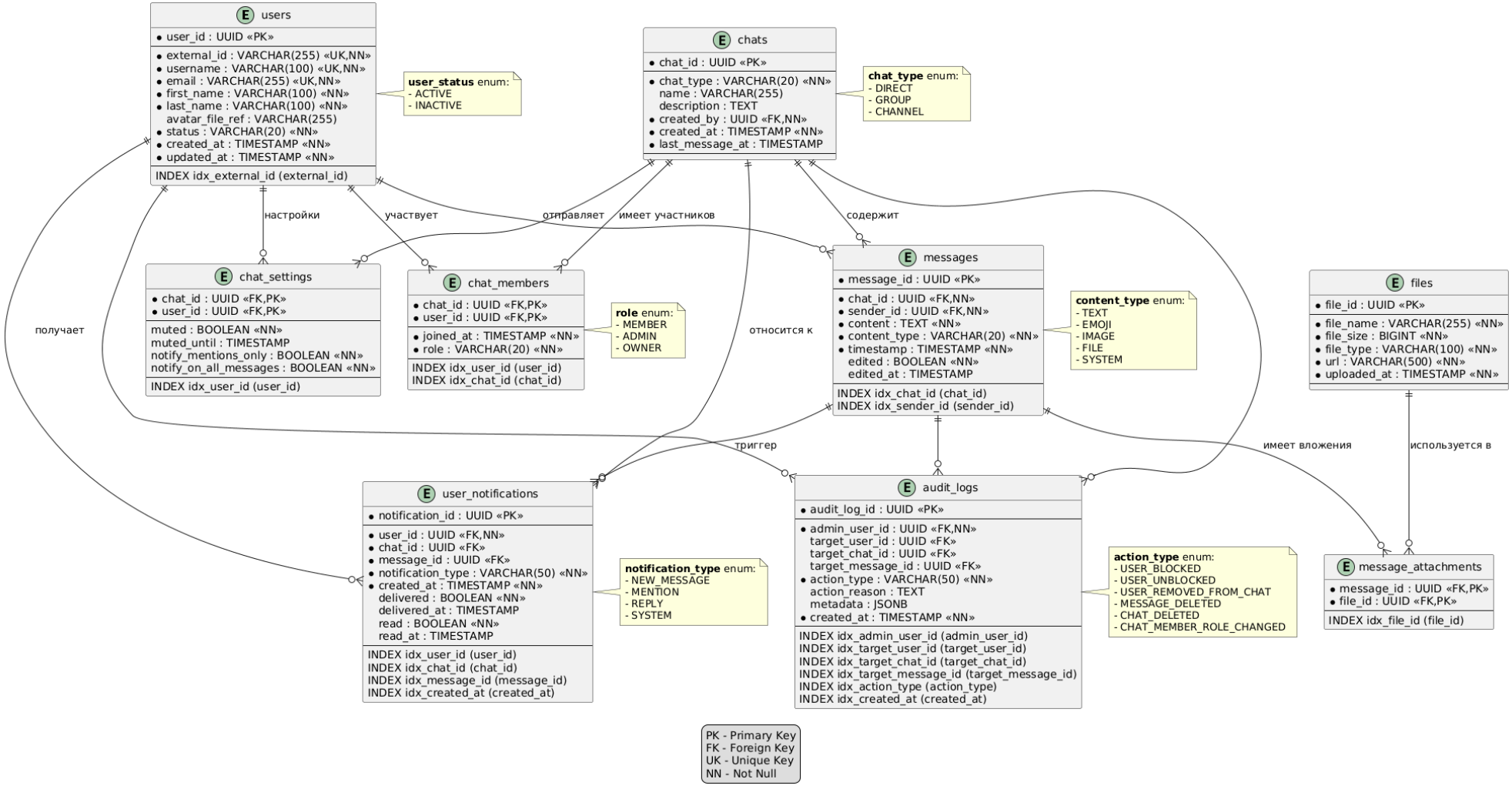
8. API/Realtime:

- REST (внешний контур) - понятный и совместимый интерфейс для клиентов и интеграций
- gRPC (внутренний контур) - более эффективные межсервисные вызовы
- WebSocket Gateway - доставка realtime-событий

9. Хранение и кэш

- PostgreSQL (primary + replicas) - транзакционное хранилище бизнес-данных
- KeyDB - presence/сессии и быстрый кэш с TTL
- Kafka - доменные события и асинхронная интеграция
- S3 (MinIO) - хранение вложений
- OpenSearch - полнотекстовый поиск

10. Модель данных



11. Безопасность и управление доступом

- Keycloak + LDAP/AD - SSO и централизованное управление учетными записями
- RBAC с проверкой владельца ресурса в каждом сервисе,
- Vault для секретов,

- HTTPS, WSS (TLS 1.3) для внешних коммуникаций
- логи событий безопасности и аудит.

12. Observability

Сбор метрик (VictoriaMetrics), дашборды и алерты (Grafana, Alertmanager)

- Технические метрики по сервисам (http_5xx_rate, ws_connect_rate, consumer_lag, service_memory_usage, service_cpu_usage, etc..)
- Продуктовые метрики (messages_sent_total, messages_delivered_total, etc…)

Логи

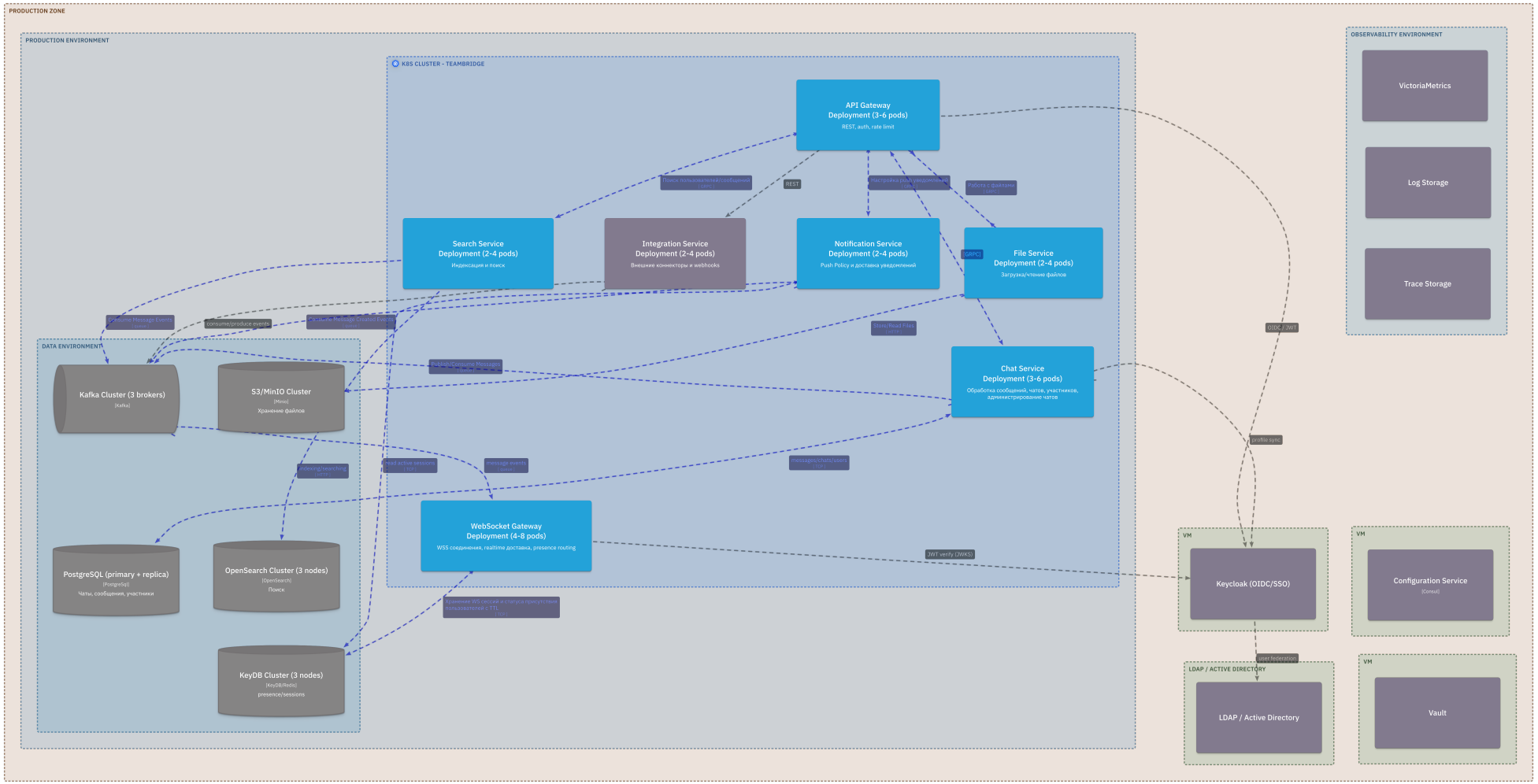
- Структурированные JSON логи во всех сервисах
- Обязательные поля - `timestamp`, `service`, `trace_id`, `request_id`, `user_id` (если допустимо), `chat_id`, `event_type`, `status`, `error_code`
- Единая схема ошибок - код, причина, классификация (`transient`/`permanent`)
- Маскирование чувствительных данных - токены, пароли, персональные данные

Трассировка

- OpenTelemetry SDK во всех backend сервисах
- Propagation контекста через HTTP и Kafka headers
- Спаны на внешние вызовы: Keycloak, PostgreSQL, Kafka, Redis, S3

13. Развёртывание

3 окружения Dev, Stage, Prod. Stateles сервисы в K8s, stateful-сервисы вне кластера; манифесты infra/k8s/, Terraform.



14. Выводы

Цель проекта достигнута: зафиксированы требования, архитектура, данные, безопасность, мониторинг и контур поставки;

https://github.com/dmordvinov/otus-system-design-homework

15. Вопросы

???

16. Спасибо!